

Actividad práctica individual

Desarrollo con Tecnologías Blockchain · Clase 04

Registro de préstamo de equipamiento en Hyperledger Fabric

Materia: Desarrollo con Tecnologías Blockchain Clase: 04 — Desarrollo con Hyperledger Fabric Fecha: 07/09/2026

Modalidad: individual Duración estimada: 90 minutos

El caso de uso

Dos laboratorios de una universidad comparten equipamiento especializado. Cada laboratorio es una organización independiente: Laboratorio de Redes (Org1) y Laboratorio de Sistemas (Org2). Cuando un equipo se transfiere de un laboratorio al otro, ambas partes deben estar de acuerdo con el registro del cambio. Ninguna organización puede modificar el estado del equipo sin el consentimiento de la otra.

El equipo `EQ-01` (un servidor de pruebas) empieza bajo responsabilidad de Org1 en el Depósito. La actividad registra su transferencia a Org2 en el Laboratorio B, y verifica que el intento de transferencia por parte de quien no es responsable actual falla con un error claro.

Actores

Actor	Rol en Fabric	Qué hace en esta actividad
Admin de Org1	Cliente firmante con identidad Org1MSP	Inicializa el ledger y transfiere EQ-01 a Org2
Admin de Org2	Cliente firmante con identidad Org2MSP	Consulta el estado final y verifica consistencia
peer0.org1	Peer endosante de Org1	Ejecuta y firma el resultado del contrato
peer0.org2	Peer endosante de Org2	Ejecuta y firma el resultado del contrato
orderer	Servicio de ordering (Raft)	Ordena las transacciones y distribuye bloques
Chaincode <code>prestamos</code>	Contrato TypeScript	Implementa las reglas de transferencia y autorización

Por qué blockchain agrega valor aquí

Sin blockchain, el registro de préstamos vive en una base de datos centralizada que una sola organización controla. Si Org1 modifica el registro, Org2 no tiene forma de verificar que el cambio fue legítimo sin confiar ciegamente en Org1.

Con Fabric:

- Ambas organizaciones ejecutan el mismo contrato sobre el mismo estado y firman el resultado antes de que se escriba. Ninguna puede alterar el historial unilateralmente.
- El ledger es inmutable: cada transferencia queda registrada con timestamp, identidad del firmante y bloque de confirmación.
- La política de endoso `AND (Org1MSP.peer, Org2MSP.peer)` es una regla técnica que la red hace cumplir automáticamente, sin depender de acuerdos informales entre las partes.

Herramientas que vamos a usar

Herramienta	Para qué
Hyperledger Fabric v2.5.16	Red blockchain permissionada con dos organizaciones

Herramienta	Para qué
fabric-samples/test-network	Red local de prueba con scripts de automatización
TypeScript + fabric-contract-api	Lenguaje del chaincode <code>prestamos</code>
fabric-chaincode-node	Runtime que ejecuta el chaincode Node.js dentro del peer
peer CLI	Cliente de línea de comandos para invocar y consultar el chaincode
Docker	Contenedores para peers, orderer y chaincode

Qué vas a poder demostrar al terminar

- La red local tiene un orderer y un peer por organización.
- El canal `equipamiento` fue creado y ambos peers participan.
- El chaincode `prestamos` fue instalado y desplegado con política AND de Org1 y Org2.
- Una transferencia requiere endoso de ambas organizaciones para confirmarse.
- El estado de EQ-01 cambia recién después de que la transacción es commiteada.
- Ambos peers ven el mismo estado final.
- Una operación firmada por quien no es responsable actual falla con el mensaje del contrato.

Requisitos

La actividad usa la red de prueba oficial de Fabric (`fabric-samples/test-network`). No se configura una red productiva.

Versiones fijadas para esta actividad: Hyperledger Fabric v2.5.16 y Fabric CA v1.5.22.

Necesitás:

- Docker funcionando.
- Git.
- cURL.
- Node.js 18 o superior y npm.
- Una terminal Bash.
- 6 GB libres de disco.

Windows — usá WSL2 con Ubuntu

Ejecutá todos los comandos dentro de la terminal de Ubuntu, no en PowerShell ni en CMD.

```
# Dentro de Ubuntu en WSL2
sudo apt-get update
sudo apt-get install -y git curl jq

# Instalar Node.js 18 con nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 18
nvm use 18
```

Linux

```
sudo apt-get update
sudo apt-get install -y git curl docker.io docker-compose-plugin jq
sudo systemctl start docker
sudo usermod -aG docker "$USER"
# Cerrá sesión y volvé a entrar, luego:
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 18
nvm use 18
```

macOS

```
brew install git curl jq node
brew install --cask docker
open /Applications/Docker.app
```

Verificación de prerequisites

Ejecutá esto y guardá la salida en tu documento de entrega:

```
docker --version
docker compose version
git --version
node --version
npm --version
```

Salida esperada (versiones mínimas):

```
Docker version 24.x o superior
Docker Compose version v2.x o superior
git version 2.x o superior
v18.x.x o superior
9.x.x o superior
```

Preparar el documento de entrega

Creá un documento llamado Clase04_Apellido_Nombre (.md, .docx o .pdf) con estos apartados:

1. Entorno y prerequisites.
2. Descarga de Fabric.
3. Compilación del chaincode.
4. Red local y canal.
5. Despliegue del chaincode.
6. Invocaciones y consultas.
7. Problemas encontrados.
8. Respuestas finales.

Cada apartado debe incluir el comando ejecutado, la salida obtenida y una explicación breve con tus palabras.

Paso 1 — Clonar este repositorio y descargar Fabric Samples, binarios e imágenes

Elegí una carpeta de trabajo sin espacios en la ruta. En Windows, trabajá dentro del filesystem de WSL (~fabric-labs), no en /mnt/c/....

```
mkdir -p ~/fabric-labs
cd ~/fabric-labs
git clone https://github.com/utn-blockchain-5k3/fabric-prestamos-equipamiento.git
cd fabric-prestamos-equipamiento

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
FABRIC_DOCKER_REGISTRY=docker.io/hyperledger ./install-fabric.sh --fabric-version 2.5.16 --ca-version 1.5.22 docker samples binary
```

La descarga tarda varios minutos dependiendo de la conexión. Cuando termina, verificá:

El script puede mostrar fabric-samples v2.5.16 does not exist, defaulting to main. No es un error: los binarios y las imágenes quedan fijados por --fabric-version 2.5.16 y --ca-version 1.5.22.

```
test -d fabric-samples/test-network && echo "fabric-samples OK"
docker images | grep hyperledger
```

Salida esperada:

```
fabric-samples OK
```

```
hyperledger/fabric-peer    2.5.16    ...
hyperledger/fabric-orderer ...
hyperledger/fabric-ca      ...
```

Si no aparece `fabric-samples` OK, el script no terminó correctamente. Repetí el comando de descarga completo.

Paso 2 — Compilar el chaincode

El chaincode de la clase ya viene incluido en este repositorio:

```
chaincode/prestamos-ts
```

Desde la raíz del repositorio, guardá la ruta absoluta en una variable:

```
cd ~/fabric-labs/fabric-prestamos-equipamiento
CHAINCODE_PATH="$PWD/chaincode/prestamos-ts"
```

Si no pudiste clonar el repositorio, también podés crear el chaincode desde cero copiando y pegando este bloque completo en la terminal:

```
mkdir -p ~/fabric-labs/prestamos-ts/src
cd ~/fabric-labs/prestamos-ts

cat > package.json <<'EOF'
{
  "name": "prestamos",
  "version": "1.0.0",
  "description": "Chaincode de préstamo de equipamiento - DTB 2026",
  "main": "dist/index.js",
  "scripts": {
    "start": "fabric-chaincode-node start",
    "build": "tsc",
    "test": "echo 'sin tests' && exit 0"
  },
  "dependencies": {
    "fabric-contract-api": "^2.2.3",
    "fabric-shim": "^2.2.3"
  },
  "devDependencies": {
    "typescript": "^5.4.5",
    "@types/node": "^20.0.0"
  }
}
EOF

cat > tsconfig.json <<'EOF'
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true,
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
EOF

cat > src/index.ts <<'EOF'
import { type Contract } from 'fabric-contract-api';
import { PrestamosContract } from './prestamos';

export const contracts: typeof Contract[] = [PrestamosContract];
EOF

cat > src/prestamos.ts <<'EOF'
import { Context, Contract } from 'fabric-contract-api';

interface Equipo {
  id: string;
  responsable: string;
  destino: string;
}

const MSP_ORG: Record<string, string> = {
```

```

    Org1: 'Org1MSP',
    Org2: 'Org2MSP',
  };

export class PrestamosContract extends Contract {

  async InitLedger(ctx: Context): Promise<void> {
    const equipo: Equipo = { id: 'EQ-01', responsable: 'Org1', destino: 'Depósito' };
    await ctx.stub.putState(equipo.id, Buffer.from(JSON.stringify(equipo)));
  }

  async ReadEquipo(ctx: Context, id: string): Promise<Equipo> {
    const data = await ctx.stub.getState(id);
    if (!data || data.length === 0) throw new Error(`el equipo ${id} no existe`);
    return JSON.parse(data.toString()) as Equipo;
  }

  async TransferirEquipo(ctx: Context, id: string, nuevoResponsable: string, nuevoDestino:
string): Promise<void> {
    const equipo = await this.ReadEquipo(ctx, id);
    const clienteMSP = ctx.clientIdentity.getMSPID();

    if (MSP_POR_ORG[equipo.responsable] !== clienteMSP) {
      throw new Error(`solo ${equipo.responsable} puede transferir el equipo ${id};
        cliente recibido: ${clienteMSP}`);
    }

    equipo.responsable = nuevoResponsable;
    equipo.destino = nuevoDestino;
    await ctx.stub.putState(id, Buffer.from(JSON.stringify(equipo)));
  }

  async GetAllEquipos(ctx: Context): Promise<Equipo[]> {
    const iterator = await ctx.stub.getStateByRange('', '');
    const equipos: Equipo[] = [];

    let result = await iterator.next();
    while (!result.done) {
      equipos.push(JSON.parse(result.value.value.toString()) as Equipo);
      result = await iterator.next();
    }
    await iterator.close();

    return equipos;
  }
}
EOF

CHAINCODE_PATH="$HOME/fabric-labs/prestamos-ts"

```

Instalá dependencias y compilá:

```

cd "$CHAINCODE_PATH"
npm install
npm run build

```

Salida esperada de npm install:

```

added 82 packages, and audited 83 packages in Xs
found 0 vulnerabilities

```

Salida esperada de npm run build:

```

> prestamos@1.0.0 build
> tsc

```

Si `tsc` no produce ningún mensaje de error, la compilación fue exitosa. Verificá que el artefacto existe:

```
test -f dist/index.js && echo "compilación OK"
```

Salida esperada:

```
compilación OK
```

Paso 3 — Levantar la red local con dos organizaciones

Volvé a la carpeta de trabajo y levanta la red:

```
cd ~/fabric-labs/fabric-samples/test-network
```

```
./network.sh down
./network.sh up createChannel -c equipamiento
```

El comando `down` limpia cualquier red previa. El comando `up createChannel` levanta los contenedores y crea el canal `equipamiento` en un solo paso.

Verificá que los tres contenedores están corriendo:

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

Salida esperada (entre otros contenedores que puedas tener):

NAMES	STATUS
peer0.org1.example.com	Up X seconds
peer0.org2.example.com	Up X seconds
orderer.example.com	Up X seconds

Si alguno de los tres no aparece, revisá la salida del comando `up` buscando líneas con `Error` o `failed`.

En tu entrega, explicá con tus palabras qué representa cada uno de estos tres contenedores y cuál es el rol de cada uno en la red.

Paso 4 — Desplegar el chaincode `prestamos`

Desde `fabric-samples/test-network`, ejecutá:

```
./network.sh deployCC \
  -c equipamiento \
  -ccn prestamos \
  -ccp "$CHAINCODE_PATH" \
  -ccl typescript \
  -ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

Este comando empaqueta el chaincode, lo instala en ambos peers, obtiene la aprobación de Org1 y Org2, y confirma la definición en el canal.

Salida esperada al final del comando:

```
Committed chaincode definition for chaincode 'prestamos' on channel 'equipamiento':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc,
Approvals: [Org1MSP: true, Org2MSP: true]
```

Verificá que el contenedor del chaincode está corriendo:

```
docker ps --format "table {{.Names}}\t{{.Status}}" | grep prestamos
```

Salida esperada:

```
dev-peer0.org1.example.com-prestamos_1.0-...    Up X seconds
dev-peer0.org2.example.com-prestamos_1.0-...    Up X seconds
```

En tu entrega, explicá qué significa la política `AND('Org1MSP.peer','Org2MSP.peer')`. ¿Qué pasaría si la política fuera `OR` en lugar de `AND`?

Paso 5 — Configurar la CLI como Org1

Desde `fabric-samples/test-network`, exportá las variables de entorno para operar como administrador de Org1:

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=${PWD}/../config/
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID=Org1MSP
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Verificá que el binario `peer` está disponible:

```
peer version
```

Salida esperada:

```
peer:
  Version: 2.5.16
  ...
```

Estas variables le dicen al binario `peer` qué identidad usar, a qué peer conectarse y cómo verificar el TLS. Si cerrás la terminal o abrís una nueva, tenés que volver a exportarlas.

Paso 6 — Inicializar el ledger

Invocá `InitLedger` para registrar EQ-01 en el estado inicial:

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/m
    sp/tlsCACerts/tlsca.example.com-cert.pem" \
  -C equipamiento \
  -n prestamos \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.e
    xample.com/tls/ca.crt" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.e
    xample.com/tls/ca.crt" \
  -c '{"function":"InitLedger","Args":[]}'
```

Salida esperada:

```
INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

El `invoke` apunta a los peers de Org1 (localhost:7051) y Org2 (localhost:9051) porque la política de endoso exige la firma de ambas organizaciones. Si apuntás solo a uno, el `invoke` falla por endoso insuficiente.

Paso 7 — Consultar el estado inicial de EQ-01

Esperá a que el commit sea visible y consultá:

```
for intento in 1 2 3 4 5; do
  peer chaincode query \
    -C equipamiento \
    -n prestamos \
    -c '{"Args":["ReadEquipo","EQ-01"]}' && break
  sleep 2
done
```

Salida esperada:

```
{"id":"EQ-01","responsable":"Org1","destino":"Depósito"}
```

Guardá esta salida en tu documento. Esto confirma que el ledger fue inicializado correctamente y que EQ-01 está bajo responsabilidad de Org1 en el Depósito.

Una consulta (`query`) no pasa por el orderer: lee directamente el World State del peer. Por eso no necesita apuntar a ambos peers ni incluir el certificado del orderer.

Paso 8 — Transferir EQ-01 a Org2

Ejecutá la transferencia. El equipo pasa de Org1 / Depósito a Org2 / Laboratorio B:

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/m
    sp/tlsCACerts/tlsca.example.com-cert.pem" \
  -C equipamiento \
  -n prestamos \
```

```
--peerAddresses localhost:7051 \
--tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.e
  xample.com/tls/ca.crt" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.e
  xample.com/tls/ca.crt" \
-c '{"function":"TransferirEquipo","Args":["EQ-01","Org2","Laboratorio B"]}'
```

Salida esperada:

```
INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Esperá a que el cambio sea visible y consultá el nuevo estado:

```
for intento in 1 2 3 4 5; do
  RESULTADO=$(peer chaincode query \
    -C equipamiento \
    -n prestamos \
    -c '{"Args":["ReadEquipo","EQ-01"]}' 2>/dev/null || true)
  echo "$RESULTADO"
  echo "$RESULTADO" | grep "Laboratorio B" && break
  sleep 2
done
```

Salida esperada:

```
{"id":"EQ-01","responsable":"Org2","destino":"Laboratorio B"}
```

Guardá ambas salidas (antes y después de la transferencia) en tu documento. La diferencia entre los dos estados es la evidencia de que la transacción fue commiteada.

Paso 9 — Consultar desde Org2

Cambiá la CLI para usar el peer de Org2:

```
export CORE_PEER_LOCALMSPID=Org2MSP
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org2.example.com/peers
  /peer0.org2.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org2.example.com/users/Adm
  in@org2.example.com/msp
export CORE_PEER_ADDRESS=localhost:9051
```

Consultá el estado desde el peer de Org2:

```
peer chaincode query \
  -C equipamiento \
  -n prestamos \
  -c '{"Args":["ReadEquipo","EQ-01"]}'
```

Salida esperada:

```
{"id":"EQ-01","responsable":"Org2","destino":"Laboratorio B"}
```

El resultado debe ser idéntico al que obtuviste desde Org1. En tu entrega, explicá por qué ambos peers tienen el mismo estado aunque son contenedores independientes.

Paso 10 — Probar una autorización rechazada

El responsable actual de EQ-01 es Org2. Volvé a configurar la CLI como Org1:

```
export CORE_PEER_LOCALMSPID=Org1MSP
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers
  /peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Adm
  in@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Intentá transferir el equipo firmando como Org1, cuando el responsable actual es Org2:

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/m
```



```

    sp/tlscacerts/tlsca.example.com-cert.pem" \
-C equipamiento \
-n prestamos \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.e
xample.com/tls/ca.crt" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.e
xample.com/tls/ca.crt" \
-c '{"function":"TransferirEquipo","Args":["EQ-01","Org2","Laboratorio C"]}'

```

Salida esperada (error):

```

Error: endorsement failure during invoke. response: status:500
message:"solo Org2 puede transferir el equipo EQ-01; cliente recibido: Org1MSP"

```

Este error lo genera el contrato durante la simulación en el peer, antes de que la transacción llegue al orderer. El peer de Org1 detecta que el MSP del cliente (Org1MSP) no coincide con el responsable actual (Org2) y rechaza el endoso.

En tu entrega, explicá la diferencia entre este rechazo y lo que pasaría si la política de endoso no se cumpliera.

Paso 11 — Limpiar la red

```

cd ~/fabric-labs/fabric-samples/test-network
./network.sh down

```

Salida esperada (fragmento):

```

Removing remaining containers
Removing generated chaincode docker images

```

Verificá que los contenedores de Fabric ya no están:

```

docker ps --format "{{.Names}}" | grep -E "peer|orderer|prestamos" || echo "red detenida OK"

```

Salida esperada:

```

red detenida OK

```

Qué debe contener tu entrega

Un único documento Clase04_Apellido_Nombre con:

- Nombre, apellido, legajo y fecha.
- Sistema operativo y versiones de Docker, Docker Compose, Git, Node.js y npm.
- Salida de fabric-samples OK y listado de imágenes Hyperledger.
- Salida de npm run build y compilación OK.
- Salida de docker ps con los tres contenedores de la red.
- Salida del deploy con Approvals: [Org1MSP: true, Org2MSP: true].
- Consulta inicial de EQ-01 → Org1 / Depósito.
- Salida del invoke de transferencia → status:200.
- Consulta posterior → Org2 / Laboratorio B.
- Consulta desde Org2 → mismo estado.
- Salida del invoke rechazado con el mensaje de error del contrato.
- Salida de network.sh down.
- Respuestas finales.
- Problemas encontrados y cómo los resolviste.

Respuestas finales

Respondé con tus palabras:

1. ¿Qué parte de la práctica corresponde a crear la red y qué parte corresponde a desplegar el chaincode?
2. ¿Por qué el invoke apunta a los peers de Org1 y Org2 al mismo tiempo?
3. ¿Qué hace el orderer durante la invocación? ¿Ejecuta el contrato?
4. ¿Por qué una consulta no necesita pasar por el orderer?
5. ¿Qué diferencia hay entre la política de endoso `AND (Org1MSP.peer, Org2MSP.peer)` y la regla de autorización implementada en `TransferirEquipo`?

Lista final de revisión

- ☐ Ejecuté todos los comandos en una terminal Bash.
- ☐ Docker estaba iniciado antes de levantar la red.
- ☐ Usé una ruta sin espacios para `fabric-labs`.
- ☐ `npm run build` terminó sin errores y generó `dist/index.js`.
- ☐ Creé el canal `equipamiento` (no `mychannel`).
- ☐ Desplugué el chaincode `prestamos` con `-ccl typescript`.
- ☐ El deploy terminó con `Approvals: [Org1MSP: true, Org2MSP: true]`.
- ☐ Invoqué `InitLedger` apuntando a ambos peers.
- ☐ Consulté `EQ-01` antes de transferir y guardé la salida.
- ☐ Invoqué `TransferirEquipo` apuntando a ambos peers.
- ☐ Consulté `EQ-01` después de transferir y guardé la salida.
- ☐ Consulté desde el peer de Org2 y guardé la salida.
- ☐ Probé la operación rechazada y guardé el mensaje de error.
- ☐ Ejecuté `./network.sh down` al finalizar.

Referencias

- Prerequisites — Hyperledger Fabric
- Install Fabric and Fabric Samples
- Using the Fabric test network
- Deploying a smart contract to a channel
- `fabric-contract-api` — npm